

# Universidade da Beira Interior



Relatório Técnico da U.C. de Sistemas Distribuídos

## **Jogo da Força Multijogador**

Sistema Cliente-Servidor em Java

Elaborado por:

**Sara Bastos - 49499**

**Juliana Ferreira - 51886**

Orientadores:

**Professor Doutor Tiago Miguel Carrola Simões**

**Professor Doutor Nuno José Matos Pereira**

Covilhã, 14 de abril de 2026

# Conteúdo

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introdução</b>                                    | <b>3</b>  |
| <b>2</b>  | <b>Objetivos</b>                                     | <b>3</b>  |
| <b>3</b>  | <b>Enquadramento Teórico</b>                         | <b>3</b>  |
| <b>4</b>  | <b>Arquitetura do Sistema</b>                        | <b>4</b>  |
| 4.1       | Servidor . . . . .                                   | 4         |
| 4.2       | Cliente . . . . .                                    | 4         |
| 4.3       | Modelo de comunicação . . . . .                      | 5         |
| <b>5</b>  | <b>Justificação da Escolha das Funções</b>           | <b>5</b>  |
| <b>6</b>  | <b>Descrição das Funções</b>                         | <b>6</b>  |
| 6.1       | Funções de comunicação . . . . .                     | 6         |
| 6.2       | Funções de processamento de mensagens . . . . .      | 6         |
| 6.3       | Funções de lógica do jogo . . . . .                  | 7         |
| 6.4       | Funções de interface . . . . .                       | 7         |
| 6.5       | Funções de apoio . . . . .                           | 7         |
| <b>7</b>  | <b>Decisões de Projeto</b>                           | <b>8</b>  |
| 7.1       | Utilização de arquitetura cliente-servidor . . . . . | 8         |
| 7.2       | Utilização do protocolo TCP . . . . .                | 8         |
| 7.3       | Mensagens em formato texto . . . . .                 | 8         |
| 7.4       | Uso de threads no servidor e no cliente . . . . .    | 8         |
| 7.5       | Sincronização por rondas . . . . .                   | 8         |
| 7.6       | Separação entre comunicação e lógica . . . . .       | 8         |
| 7.7       | Timeout por rondas . . . . .                         | 9         |
| <b>8</b>  | <b>Funcionamento do Sistema</b>                      | <b>9</b>  |
| 8.1       | Inicialização do servidor . . . . .                  | 9         |
| 8.2       | Ligação dos clientes . . . . .                       | 9         |
| 8.3       | Fase de lobby . . . . .                              | 9         |
| 8.4       | Início do jogo . . . . .                             | 10        |
| 8.5       | Execução das rondas . . . . .                        | 10        |
| 8.6       | Final do jogo . . . . .                              | 10        |
| <b>9</b>  | <b>Explicação da Execução do Programa</b>            | <b>10</b> |
| 9.1       | Compilação . . . . .                                 | 11        |
| 9.2       | Execução . . . . .                                   | 11        |
| 9.3       | Condições necessárias . . . . .                      | 11        |
| <b>10</b> | <b>Exemplo de Fluxo de Utilização</b>                | <b>11</b> |

|                                    |           |
|------------------------------------|-----------|
| <b>11 Resultados Obtidos</b>       | <b>12</b> |
| <b>12 Dificuldades Encontradas</b> | <b>12</b> |
| <b>13 Melhorias Futuras</b>        | <b>13</b> |
| <b>14 Conclusão</b>                | <b>13</b> |

# 1 Introdução

Este relatório descreve o desenvolvimento de um sistema distribuído baseado no clássico jogo da forca, implementado através de uma arquitetura cliente-servidor com comunicação por sockets TCP/IP.

O principal objetivo do projeto consiste em permitir a participação simultânea de vários jogadores num jogo síncrono, no qual o estado é controlado centralmente pelo servidor e partilhado com todos os clientes ligados. Desta forma, garante-se consistência na execução, coordenação entre participantes e fiabilidade na comunicação.

Para além da vertente prática de implementação, este projeto permitiu aplicar conceitos fundamentais da unidade curricular, nomeadamente comunicação em rede, protocolos de transporte, sincronização entre processos e organização modular de software.

Ao longo deste relatório são apresentados os objetivos do trabalho, a arquitetura do sistema, a justificação das funções utilizadas, as principais decisões tomadas durante o desenvolvimento, o funcionamento geral da aplicação, a forma de execução e uma análise final do trabalho realizado.

## 2 Objetivos

Os principais objetivos deste projeto foram os seguintes:

- Desenvolver um sistema cliente-servidor funcional;
- Implementar comunicação entre processos através de sockets TCP/IP;
- Garantir sincronização entre múltiplos jogadores;
- Aplicar conceitos de programação concorrente;
- Estruturar o código de forma modular, clara e reutilizável;
- Simular um ambiente multijogador controlado por um servidor central;
- Assegurar o envio e receção correta de mensagens entre clientes e servidor;
- Implementar um jogo com regras simples, mas com controlo rigoroso do estado.

## 3 Enquadramento Teórico

A arquitetura cliente-servidor é um dos modelos mais utilizados no desenvolvimento de aplicações distribuídas. Neste modelo, existe uma entidade central, designada por servidor, responsável por fornecer serviços ou gerir recursos, enquanto várias entidades cliente estabelecem ligação com esse servidor para solicitar operações ou trocar informação de forma estruturada.

Neste projeto, esta arquitetura permite centralizar o controlo do jogo e garantir consistência entre todos os jogadores.

No contexto deste projeto, o servidor é responsável por controlar toda a lógica do jogo da forca, incluindo a escolha da palavra, a gestão das jogadas e a verificação das condições de vitória ou derrota. Os clientes funcionam como interfaces de interação com os jogadores, permitindo enviar jogadas e receber atualizações do estado do jogo.

A comunicação é efetuada através do protocolo TCP/IP. O protocolo TCP foi escolhido por oferecer uma transmissão fiável, orientada à ligação e com controlo de ordem dos dados. Isto significa que as mensagens enviadas chegam ao destino pela ordem correta e sem perdas de informação, o que é essencial num jogo multijogador sincronizado.

Outro conceito importante neste projeto é o de concorrência. Como o servidor pode ter vários clientes ligados ao mesmo tempo, é necessário gerir múltiplas interações em simultâneo.

No servidor, cada cliente é tratado através de uma thread independente, permitindo a gestão simultânea de múltiplas ligações.

Da mesma forma, no cliente, é útil separar a receção de mensagens da entrada de dados do utilizador, recorrendo a threads para evitar bloqueios.

## 4 Arquitetura do Sistema

O sistema implementado segue o modelo cliente-servidor, no qual existe uma clara separação entre a entidade que gere a lógica do jogo e as entidades que representam os jogadores.

### 4.1 Servidor

O servidor é o componente central do sistema. É nele que reside toda a lógica associada ao jogo da forca. As suas principais responsabilidades são:

- Aceitar ligações de múltiplos clientes;
- Registar os jogadores participantes;
- Gerir o lobby inicial;
- Selecionar a palavra do jogo;
- Controlar o número de tentativas disponíveis;
- Recolher as jogadas dos clientes;
- Atualizar o estado do jogo;
- Enviar informação atualizada a todos os jogadores;
- Determinar o final do jogo.

O servidor assume assim o papel de coordenador, garantindo que todos os jogadores observam o mesmo estado do jogo e que as regras são aplicadas de forma uniforme. Cada cliente é tratado através de uma thread independente, permitindo ao servidor gerir múltiplos jogadores em simultâneo.

### 4.2 Cliente

O cliente representa a aplicação executada por cada jogador. O seu papel é mais simples do que o do servidor, sendo orientado para a interação com o utilizador.

As principais funções do cliente são:

- Estabelecer ligação com o servidor;
- Receber mensagens e instruções;
- Apresentar o estado atual do jogo na consola;
- Ler a jogada introduzida pelo utilizador;
- Enviar essa jogada para o servidor;
- Mostrar mensagens de vitória, derrota ou erro.

### 4.3 Modelo de comunicação

A comunicação entre cliente e servidor é baseada em mensagens de texto simples, normalmente enviadas uma por linha. Este tipo de abordagem facilita a implementação, leitura e depuração do protocolo.

Exemplos de mensagens possíveis:

- WELCOME <id> <players<sub>total</sub>>
- START <mask> <attempts> <timeout>
- ROUND <k> <mask> <attempts> <used<sub>letters</sub>>
- STATE <mask> <attempts> <used<sub>letters</sub>>
- END WIN <ids> <word>
- END <word>

Este modelo permite que cada mensagem tenha um significado bem definido, sendo depois interpretada pelas funções adequadas em cada lado da aplicação.

## 5 Justificação da Escolha das Funções

Durante o desenvolvimento do projeto, optou-se por estruturar a aplicação em várias funções com responsabilidades específicas. Esta decisão teve como base os princípios da programação modular, que defendem a divisão de um programa em componentes menores e logicamente independentes.

A utilização de funções distintas oferece várias vantagens:

- melhora a organização geral do código;
- facilita a leitura e compreensão do programa;
- simplifica a manutenção e correção de erros;
- permite reutilizar partes do código;
- torna a implementação mais próxima da lógica real do problema.

Num sistema distribuído, esta separação é ainda mais importante, porque existem várias tarefas distintas a decorrer: comunicação, processamento das mensagens, atualização do estado do jogo e apresentação de resultados. Misturar todas estas responsabilidades num único bloco de código tornaria o projeto mais difícil de desenvolver e de testar.

Assim, as funções foram escolhidas e distribuídas de acordo com o seu papel dentro do sistema.

## 6 Descrição das Funções

Nesta secção são descritos os principais grupos de funções utilizados no projeto.

### 6.1 Funções de comunicação

As funções de comunicação são responsáveis pela troca de mensagens entre o cliente e o servidor. Estas funções permitem criar as ligações, enviar informação e receber dados vindos da rede.

Entre as operações mais importantes incluem-se:

- criação do socket do servidor;
- aceitação de novas ligações;
- criação do socket do cliente;
- envio de mensagens através de `PrintWriter`;
- receção de mensagens através de `BufferedReader`.

Estas operações são implementadas através das classes `ServerSocket` e `Socket` no servidor, e `Socket` no cliente.

### 6.2 Funções de processamento de mensagens

Depois de recebidas, as mensagens têm de ser interpretadas. Para isso, existem funções que analisam o conteúdo das strings recebidas e identificam o seu significado.

Estas funções podem:

- verificar o tipo de comando recebido;
- separar a mensagem em partes;
- extrair argumentos;
- encaminhar a informação para a lógica apropriada.

Por exemplo, quando o cliente recebe uma mensagem **START**, deve reconhecer que o jogo começou e atualizar a consola com a palavra escondida e o número de tentativas.

## 6.3 Funções de lógica do jogo

Estas funções implementam as regras do jogo da forca. São responsáveis por controlar toda a evolução da partida.

Entre as principais tarefas encontram-se:

- escolher uma palavra aleatória;
- representar a palavra escondida;
- verificar se a letra jogada existe na palavra;
- atualizar a palavra parcialmente descoberta;
- diminuir o número de tentativas em caso de erro;
- verificar se a palavra foi totalmente descoberta;
- verificar se o número de tentativas chegou a zero.

Estas são as funções mais importantes do ponto de vista funcional, pois determinam o comportamento real do jogo.

## 6.4 Funções de interface

Embora o projeto seja baseado em consola, é necessário apresentar informação ao utilizador de forma clara. Para isso, existem funções que ajudam a formatar e imprimir mensagens no terminal.

Estas funções permitem:

- mostrar mensagens de boas-vindas;
- apresentar a palavra com letras escondidas;
- indicar o número de tentativas disponíveis;
- informar o jogador sobre vitória ou derrota;
- tornar a leitura da consola mais organizada.

## 6.5 Funções de apoio

Para além das funções principais, podem existir funções auxiliares, por exemplo para:

- formatar a palavra;
- validar entradas;
- limpar ou preparar strings;
- gerir listas de jogadores;
- simplificar tarefas repetidas.

Estas funções não representam diretamente uma funcionalidade principal, mas ajudam a tornar o código mais limpo e mais fácil de manter.



## 7 Decisões de Projeto

Ao longo do desenvolvimento, foram tomadas várias decisões relevantes para a implementação final do sistema.

### 7.1 Utilização de arquitetura cliente-servidor

A escolha desta arquitetura foi natural, uma vez que o problema envolve múltiplos utilizadores e um estado comum do jogo. Ao centralizar a lógica no servidor, foi possível garantir consistência, controlo e sincronização entre todos os participantes.

### 7.2 Utilização do protocolo TCP

Foi escolhido o protocolo TCP em vez de UDP devido à sua fiabilidade. Num jogo em que cada mensagem pode alterar o estado global, seria inadequado utilizar um protocolo sem garantias de entrega ou de ordem.

### 7.3 Mensagens em formato texto

Optou-se por usar mensagens simples em texto, uma por linha, porque este formato é fácil de implementar e testar. Além disso, torna possível observar diretamente a comunicação trocada entre cliente e servidor, o que ajuda bastante na fase de depuração.

### 7.4 Uso de threads no servidor e no cliente

No servidor, cada cliente é tratado através de uma thread independente, permitindo gerir múltiplas ligações em simultâneo.

No cliente, uma thread secundária pode ser usada para ouvir continuamente o servidor, enquanto a thread principal fica responsável pela leitura de input do utilizador. Esta decisão evita bloqueios e melhora o comportamento da aplicação.

### 7.5 Sincronização por rondas

Uma decisão importante foi o facto de o servidor apenas atualizar o estado do jogo depois de recolher todas as jogadas da ronda. Esta abordagem assegura que todos os jogadores participam em igualdade de circunstâncias e que o estado do jogo evolui de forma sincronizada.

### 7.6 Separação entre comunicação e lógica

Sempre que possível, a comunicação em rede foi mantida separada da lógica do jogo. Esta decisão melhora a estrutura do código e facilita futuras alterações, como por exemplo substituir a consola por uma interface gráfica.

## 7.7 Timeout por rondas

Foi implementado um timeout por ronda, permitindo que o jogo avance mesmo quando um ou mais jogadores não submetem a sua jogada. Esta decisão garante fluidez na execução do jogo e evita bloqueios.

# 8 Funcionamento do Sistema

O funcionamento do sistema pode ser descrito em várias fases.

## 8.1 Inicialização do servidor

O servidor é iniciado primeiro. Nesta fase:

- é criado o socket servidor;
- é definida a porta de escuta;
- o servidor entra em espera por novas ligações.

## 8.2 Ligação dos clientes

Cada cliente, quando executado, tenta estabelecer ligação ao servidor através do endereço IP e da porta definidos. Quando a ligação é bem-sucedida:

- o cliente passa a estar registado;
- o servidor pode enviar uma mensagem inicial de boas-vindas;
- o jogador fica à espera do início do jogo.

## 8.3 Fase de lobby

Antes de iniciar o jogo, o servidor aguarda até existir o número mínimo de jogadores. Caso o número mínimo de jogadores não seja atingido, o servidor pode iniciar o jogo após um tempo limite de espera definido (timeout).

Durante esta fase, podem ocorrer operações como:

- contagem do número de jogadores ligados;
- atribuição de identificadores;
- envio de mensagens de preparação.

## 8.4 Início do jogo

Quando as condições estão reunidas:

- o servidor escolhe aleatoriamente uma palavra;
- define o número inicial de tentativas;
- cria a representação inicial da palavra escondida;
- envia o estado inicial a todos os clientes através de uma mensagem START

## 8.5 Execução das rondas

Cada ronda do jogo segue, em termos gerais, o seguinte processo:

1. cada cliente recebe o estado atual do jogo através de uma mensagem ROUND;
2. o jogador introduz a sua jogada;
3. o cliente envia a jogada para o servidor;
4. o servidor recolhe as jogadas recebidas;
5. a lógica do jogo processa essas jogadas;
6. o estado é atualizado;
7. o novo estado é enviado novamente para todos os clientes através de uma mensagem STATE.

## 8.6 Final do jogo

O jogo termina quando ocorre uma das seguintes situações:

- a palavra é completamente descoberta;
- o número de tentativas chega a zero.

No final, o servidor envia uma mensagem de término aos clientes, indicando o resultado da partida. Após isso, as ligações são encerradas pelo servidor.

# 9 Explicação da Execução do Programa

Para a correta execução do sistema, é necessário seguir uma ordem específica. O sistema deve ser executado num ambiente que suporte Java e comunicação em rede local.

Em primeiro lugar, deve ser compilado e executado o servidor. Só depois é que os clientes podem ser iniciados, uma vez que estes dependem de um servidor já ativo para estabelecer ligação.

## 9.1 Compilação

A compilação transforma os ficheiros `.java` em ficheiros `.class`, que podem depois ser executados pela máquina virtual Java.

Exemplo de compilação:

```
javac server/.java client/.java
```

## 9.2 Execução

Depois de compilados, os programas podem ser executados da seguinte forma.

Execução do servidor:

```
java server.HangmanServer
```

Execução do cliente (deve ser executado em múltiplas instâncias):

```
java client.HangmanClient
```

Se existirem packages definidos no projeto, os comandos podem ter de ser ajustados de acordo com a estrutura dos diretórios e o nome completo das classes.

A execução pode também ser realizada através de um ambiente de desenvolvimento como o Eclipse, facilitando a compilação e execução das classes.

## 9.3 Condições necessárias

Para que a execução seja bem-sucedida, devem verificar-se algumas condições:

- o servidor deve estar ativo antes dos clientes;
- os clientes devem usar o endereço IP correto;
- a porta configurada deve ser a mesma no cliente e no servidor;
- deve existir pelo menos dois clientes para iniciar o jogo;
- não deve existir bloqueio da porta por firewall ou outro processo.

## 10 Exemplo de Fluxo de Utilização

Um exemplo típico de utilização do sistema pode ser descrito da seguinte forma:

1. o servidor é iniciado;
2. dois ou mais clientes executam a aplicação;
3. os clientes ligam-se ao servidor;
4. o servidor inicia o jogo e envia o estado inicial através de uma mensagem START
5. cada jogador introduz uma letra ou uma palavra;

6. o servidor processa as jogadas de cada ronda e envia atualizações através das mensagens ROUND e STATE;
7. o jogo termina com vitória (palavra descoberta) ou derrota (tentativas esgotadas);
8. o servidor envia a mensagem final e encerra as ligações com os clientes.

Este fluxo demonstra o comportamento normal esperado do sistema em funcionamento.

## 11 Resultados Obtidos

O sistema desenvolvido permitiu a implementação de um jogo da forca multijogador funcional, com comunicação entre cliente e servidor através de sockets TCP/IP.

Durante os testes realizados, verificou-se que:

- os clientes conseguem estabelecer ligação com o servidor;
- as mensagens são transmitidas corretamente e de acordo com o protocolo definido;
- o estado do jogo é mantido de forma consistente entre todos os participantes;
- os jogadores recebem atualizações sincronizadas em cada ronda;
- o sistema consegue identificar corretamente situações de vitória e derrota..

Os resultados obtidos demonstram que a arquitetura adotada foi adequada ao problema e que os principais objetivos definidos foram atingidos com sucesso.

## 12 Dificuldades Encontradas

Durante o desenvolvimento deste projeto, algumas dificuldades puderam surgir, especialmente em aspetos ligados à comunicação e sincronização.

Entre as dificuldades mais relevantes destacam-se:

- gestão simultânea de vários clientes;
- interpretação correta das mensagens recebidas;
- sincronização das jogadas por rondas;
- tratamento de erros de ligação;
- organização do código de forma modular.

Estas dificuldades foram ultrapassadas através de testes sucessivos, separação clara de responsabilidades e validação progressiva do funcionamento do sistema.

## 13 Melhorias Futuras

Embora o sistema desenvolvido cumpra os objetivos principais, existem várias melhorias que poderiam vir a ser implementadas no futuro:

- criação de uma interface gráfica;
- suporte para mais jogadores;
- adição de sistema de pontuação;
- armazenamento de resultados em ficheiro ou base de dados;
- autenticação de jogadores;
- melhoria do protocolo de mensagens;
- tratamento mais avançado de exceções;
- possibilidade de várias partidas consecutivas sem reiniciar a aplicação.

Estas melhorias tornariam o sistema mais completo, mais robusto e mais próximo de uma aplicação real.

## 14 Conclusão

Este projeto permitiu consolidar conhecimentos importantes nas áreas de redes de computadores, programação concorrente e sistemas distribuídos.

A implementação de um jogo da forca multijogador em arquitetura cliente-servidor revelou-se um exercício prático bastante útil para compreender o funcionamento da comunicação em rede, a importância da sincronização entre processos e a necessidade de estruturar o código de forma modular.

A escolha do protocolo TCP, a centralização da lógica do jogo no servidor e a separação entre comunicação, processamento e interface mostraram-se adequadas para cumprir os requisitos do trabalho.

De uma forma geral, considera-se que o projeto atingiu os objetivos inicialmente propostos, resultando numa aplicação funcional, organizada e compatível com os princípios abordados na unidade curricular.